

Programma aziendale React.js

Percorso intermedio - avanzato per due partecipanti

Durata	20 ore
Partecipanti	2 persone
Livello in ingresso	Intermedio
Formato consigliato	5 incontri da 4 ore oppure 10 incontri da 2 ore
Approccio	Formazione operativa, laboratorio guidato, code review e project work

Obiettivo generale

Il percorso accompagna due partecipanti con competenze intermedie nello sviluppo React verso una padronanza avanzata di architettura frontend, gestione dello stato, performance, form enterprise, testing e preparazione al rilascio. Il taglio è aziendale: ogni argomento viene collegato a scenari reali come dashboard, aree riservate, flussi autorizzativi, moduli complessi e integrazione con servizi API.

Documento redatto come schema formativo approfondito, utilizzabile per proposta didattica, preventivo, catalogo corsi o piano interno di upskilling.

1. Destinatari e prerequisiti

Il corso è progettato per sviluppatori frontend, full stack developer o figure tecniche aziendali che utilizzano già React in progetti reali e desiderano consolidare un metodo di lavoro più strutturato, scalabile e vicino alle esigenze enterprise.

Prerequisiti consigliati

- Buona conoscenza di JavaScript moderno: funzioni, moduli, destructuring, promise, async/await.
- Esperienza con componenti funzionali React e principali Hooks: useState, useEffect, useRef.
- Conoscenza base di routing, chiamate HTTP e gestione di dati provenienti da API.
- Familiarità con Git, npm o pnpm, ambiente di sviluppo con Node.js e editor come Visual Studio Code.
- Nozioni di CSS o utilizzo di librerie UI aziendali.

2. Competenze in uscita

- Progettare componenti riutilizzabili, leggibili e manutenibili.
- Distinguere correttamente client state, server state, derived state e UI state.
- Applicare pattern avanzati con custom hooks, composition, reducer e context selettivi.
- Ottimizzare rendering e caricamento tramite memoizzazione, lazy loading e profiling.
- Gestire form complessi, validazione, accessibilità ed error handling coerente.
- Scrivere test mirati e predisporre il codice per build, code review e deploy.
- Ragionare in termini di architettura, non solo di singoli componenti.

3. Struttura del percorso

Modulo	Tema	Durata	Focus operativo
1	Architettura React moderna	4 ore	Component design, custom hooks, struttura progetto
2	State management avanzato	4 ore	Client state, server state, cache, reducer
3	Performance e rendering	4 ore	Profiling, memoizzazione, code splitting
4	Form enterprise e UX	4 ore	Form complessi, validazione, accessibilità
5	Testing, qualità e rilascio	4 ore	Test, review, build, project work

Modulo 1 - Architettura React moderna

Durata: 4 ore

Contenuti didattici

- Ripasso mirato dei concetti React utili nei progetti complessi.
- Component composition: quando scomporre, quando aggregare, come evitare componenti monolitici.
- Presentational components, container components e separazione tra logica e interfaccia.
- Custom hooks per isolare logica di dominio, effetti collaterali e accesso ai dati.
- Gestione delle dipendenze tra componenti e riduzione del prop drilling.
- Organizzazione cartelle: feature based structure, shared components, services, hooks, utils.
- Naming convention, standard di progetto e leggibilità del codice in team.

Esercitazione pratica

Refactoring guidato di una sezione applicativa: da componenti troppo grandi a struttura modulare con hook dedicati e componenti riutilizzabili.

Output del modulo

Checklist architetturale per valutare componenti e cartelle di un progetto React.

Modulo 2 - State management avanzato

Durata: 4 ore

Contenuti didattici

- Differenza tra stato locale, globale, derivato, persistente e server state.
- Context API: vantaggi, limiti, suddivisione dei contesti e prevenzione dei render inutili.
- useReducer per logiche di stato prevedibili e transizioni esplicite.
- Pattern per loading, error, empty state, retry e aggiornamento asincrono.
- Data fetching: separazione tra accesso API, trasformazione dati e rendering.
- Caching, invalidazione, refresh dati e sincronizzazione con backend.
- Optimistic update e gestione rollback in caso di errore.

Esercitazione pratica

Creazione di una dashboard con filtri, lista dati, chiamate API simulate, aggiornamento ottimistico e gestione coerente degli stati di interfaccia.

Output del modulo

Mappa decisionale per scegliere dove collocare lo stato e come strutturare i reducer.

Modulo 3 - Performance, rendering e scalabilita

Durata: 4 ore

Contenuti didattici

- Meccanismi di rendering e re-render in React.
- Identificazione delle cause piu frequenti di lentezza: props instabili, context troppo ampi, calcoli pesanti.
- Uso corretto di React.memo, useMemo e useCallback: benefici, costi e anti-pattern.
- Lazy loading dei componenti e code splitting per sezioni applicative.
- Suspense e gestione del caricamento progressivo.
- Virtualizzazione concettuale di liste e tabelle con molti dati.
- Analisi con React DevTools Profiler e lettura dei risultati.

Esercitazione pratica

Profiling di una pagina lenta, individuazione dei colli di bottiglia e applicazione di ottimizzazioni misurabili.

Output del modulo

Report sintetico prima/dopo con motivazione tecnica degli interventi.

Modulo 4 - Form enterprise, UX e accessibilita

Durata: 4 ore

Contenuti didattici

- Pattern per form semplici, form complessi e form multi-step.
- Controlled e uncontrolled inputs: quando utilizzare ciascun approccio.
- Validazione client-side, messaggi di errore e feedback utente.
- Gestione di submit asincroni, salvataggio parziale e prevenzione doppio invio.
- Componenti enterprise: tabelle, modali, wizard, tab, filtri avanzati.
- Accessibilita pratica: label, focus management, tastiera, stati ARIA essenziali.
- Gestione di ruoli e permessi lato frontend: visibilita, abilitazione e protezione delle azioni.

Esercitazione pratica

Sviluppo di un form aziendale multi-step con validazione, riepilogo finale, gestione errori e differenziazione UI in base al ruolo utente.

Output del modulo

Template di form riusabile con schema di validazione e UX coerente.

Modulo 5 - Testing, qualita, React avanzato e rilascio

Durata: 4 ore

Contenuti didattici

- Testing strategy: cosa testare, cosa non testare, priorit  nei progetti aziendali.
- Unit test su funzioni, hook e componenti isolati.
- Integration test su flussi utente: compilazione form, filtri, chiamate simulate.
- Code review: leggibilit , naming, responsabilit  dei componenti, rischi architetturali.
- Build di produzione, variabili ambiente, gestione configurazioni.
- Introduzione ai pattern React moderni: transizioni, stato ottimistico e gestione avanzata delle operazioni asincrone.
- Preparazione del project work finale e revisione tecnica conclusiva.

Esercitazione pratica

Completamento di una mini-app aziendale con routing, gestione dati, form, ottimizzazioni e test essenziali.

Output del modulo

Mini progetto dimostrativo e checklist di rilascio frontend.

4. Project work finale

Durante il percorso i partecipanti costruiscono progressivamente una mini applicazione aziendale. Il progetto ha funzione didattica, ma simula esigenze reali: autenticazione semplificata, dashboard dati, gestione form, stati asincroni, validazione, ottimizzazione e test.

- Routing tra aree funzionali: dashboard, dettaglio elemento, creazione o modifica record.
- Component library interna con bottoni, card, campi form, messaggi di stato e layout.
- Servizio dati centralizzato con gestione di loading, error, retry e empty state.
- Form multi-step con validazione e salvataggio controllato.
- Gestione ruoli semplificata per mostrare o disabilitare azioni.
- Ottimizzazioni su rendering e caricamento dei componenti.
- Test essenziali su hook, componenti e flussi principali.

5. Metodologia didattica

La metodologia privilegia l'apprendimento operativo. Ogni concetto viene introdotto con una spiegazione sintetica, seguito da esempi guidati e da applicazione pratica sul progetto. Con due partecipanti è possibile mantenere un elevato livello di personalizzazione: il docente può adattare esempi, ritmo e approfondimenti ai framework, alle librerie UI e alle pratiche già utilizzate in azienda.

Fase	Descrizione
Brief tecnico	Analisi del livello dei partecipanti, strumenti usati e obiettivi applicativi.
Dimostrazione	Il docente mostra pattern e implementazioni su esempi concreti.
Laboratorio	I partecipanti implementano funzionalità assistiti dal docente.
Code review	Revisione ragionata delle scelte tecniche, naming e architettura.
Consolidamento	Checklist finale, riepilogo degli errori comuni e indicazioni operative.

6. Materiali e strumenti

- Slide tecniche sintetiche e consultabili dopo il corso.
- Repository Git con esempi, esercizi, branch progressivi e project work.
- Checklist di architettura React e code review.
- Template di componenti, hook, servizi API e form.
- Ambiente consigliato: Node.js LTS, npm o pnpm, Visual Studio Code, browser con React DevTools.

7. Valutazione e risultati attesi

La valutazione è basata sull'osservazione delle esercitazioni, sulla qualità del project work e sulla capacità dei partecipanti di motivare le proprie scelte tecniche. Il risultato atteso non è solo la conoscenza di singole API React, ma l'acquisizione di un metodo riutilizzabile nei progetti aziendali.

- Corretta scomposizione dei componenti.
- Gestione coerente dello stato e dei dati asincroni.
- Capacità di individuare e ridurre re-render non necessari.
- Qualità dell'esperienza utente nei form e negli stati applicativi.
- Presenza di test significativi e codice leggibile.
- Aderenza a convenzioni condivise e facilità di manutenzione.

8. Calendario suggerito

Incontro	Durata	Argomenti principali
1	4 ore	Architettura, component design, custom hooks
2	4 ore	State management, reducer, server state
3	4 ore	Performance, memoizzazione, profiling
4	4 ore	Form enterprise, UX, accessibilità, ruoli
5	4 ore	Testing, build, project work, revisione finale

Conclusione

Il percorso fornisce una base solida per elevare la qualità dello sviluppo React in azienda. La combinazione tra teoria mirata, laboratorio e revisione del codice permette ai partecipanti di applicare immediatamente le competenze acquisite su progetti reali, riducendo debito tecnico, inconsistenze architetturali e problemi di performance.